PROJECT REPORT

LOAD-SHEDDING TIME MANAGER FOR RURAL AREAS

Submitted by: Angel Grace Prince

Registered Email ID:angelgraceprince3333@gmail.com

KNR24EC018

Department: ECE

Government College Of Engineering Kannur

Academic Year 2024–2028

Abstract

Load shedding is a scheduled interruption of electrical power supply that can significantly affect the daily activities of people living in rural areas. Lack of timely and easily accessible information about power outages can create difficulties for households, students, farmers, and small businesses.

The proposed **Load-Shedding Time Manager for Rural Areas** is a web-based system designed to manage and display scheduled power outages. The system provides users with access to daily and weekly load-shedding schedules and allows schedules to be organized according to different regions.

The project uses a web-based architecture consisting of an HTML, CSS, and JavaScript frontend and a Python Flask backend. An administrative interface is provided for entering and updating outage schedules. The system can also be extended with time-based alerts, SMS/push notifications, offline caching, and optional hardware displays.

The implemented Flask prototype demonstrates the basic functionality of managing and displaying load-shedding information through a simple web interface. The system improves transparency and allows users to plan their activities around scheduled electricity interruptions.

Contents

1 Introduction **6**

1.1 Problem Statement 6

2 Objectives **7**

3 Literature Review and Existing System **8**

3.1 Existing System 8

3.2 Limitations of Existing System 8

3.3 Proposed System 8

3.4 Comparison 9

4 Methodology **10**

4.1 System Architecture 10

4.2 Technologies Used 11

4.3 Working Procedure 11

4.4 Functional Modules 11

4.4.1 User Module 11

4.4.2 Schedule Module 12

4.4.3 Admin Module 12

4.4.4 Region Module 12

4.4.5 Notification Extension 12

5 Implementation and Code **13**

5.1 Project Structure 13

5.2 Flask Backend 13

5.2.1 Required Libraries 13

5.2.2 Flask Application 13

5.2.3 Home Page 14

5.2.4 Schedule Data 14

5.3 Complete Backend Code 14

5.4 Frontend Code 21

5.4.1 HTML 21

5.4.2 CSS 28
5.5 Administrative Interface 37

6 Output Snapshots **44**
6.1 Main Dashboard 44
6.2 Administrative Panel 45
6.3 Schedule Added Successfully 46
6.4 Running Flask Server 46

7 Result and Analysis **48**
7.1 Result 48
7.2 Functional Testing 48
7.3 Analysis 48

8 Conclusion **50**

List of Figures

6.1 Load-Shedding Time Manager Main Dashboard 45
6.2 Administrative Panel 46
6.3 Updated Load-Shedding Schedule 46
6.4 Flask Server Running in VS Code Terminal 47

List of Tables

3.1 Comparison of Existing and Proposed Systems 9

4.1 Technologies Used 11

7.1 Functional Test Results 48

Chapter 1 Introduction

Electricity is an essential requirement for households, education, agriculture, communication, and small businesses. In areas where scheduled power interruptions occur, users need reliable information about the expected outage periods so that important activities can be planned accordingly.

The project proposed in this report is a **Load-Shedding Time Manager for Rural Areas**. Its main purpose is to provide residents and small businesses with accessible information about scheduled electricity outages.

The project description specifies a system that manages and displays scheduled power outages and helps rural users plan their daily activities during periods of electricity unavailability.

The proposed system provides a web-based interface through which users can view schedules. An administrative interface can be used by authorized personnel to enter or update outage information for different regions.

The system is implemented as a Flask-based web application for the software demonstration. The architecture can also be extended to support databases, notifications, and optional hardware displays.

1.1 Problem Statement

Scheduled power outages can cause inconvenience when users do not have clear information about the expected timing and duration of power interruptions.

The problem addressed by this project is the lack of a simple, centralized system through which rural users can obtain region-specific load-shedding schedules.

The project therefore aims to provide an organized digital platform for managing and displaying scheduled outage information.

Chapter 2 Objectives

The main objective of the project is to develop a system that manages and displays scheduled power outages in rural areas.

The specific objectives are:

1. To develop a simple interface for displaying load-shedding schedules.
2. To provide daily and weekly outage information to users.
3. To support region-wise schedules and filtering.
4. To provide an administrative interface for entering and updating outage times.
5. To display relevant information such as outage start time, end time, region, and reason.
6. To provide a foundation for implementing time-based alerts before scheduled power cuts.
7. To develop the software prototype using Python Flask, HTML, CSS, and JavaScript.
8. To provide a system that can be extended with databases, notifications, and optional offline hardware displays.

Chapter 3 Literature Review and Existing System

3.1 Existing System

In a conventional load-shedding environment, users may receive information through announcements, messages, notices, or other communication channels. Such approaches may not always provide region-specific information in a convenient and centralized format.

Users may therefore have difficulty determining:

- When the next outage will occur.
- How long the outage is expected to continue.
- Whether a particular schedule applies to their region.
- Whether the schedule has been updated.

3.2 Limitations of Existing System

The major limitations that motivate the proposed system include:

1. Lack of a centralized schedule interface.
2. Difficulty in accessing region-specific information.
3. Limited ability to update schedules dynamically.
4. Lack of a dedicated administrative interface.
5. Limited support for automated alerts.

3.3 Proposed System

The proposed Load-Shedding Time Manager addresses these limitations by providing a web-based platform for managing and displaying scheduled outages.

The project specification identifies HTML/CSS/JavaScript or ReactJS for the frontend and Python Flask, Node.js, or Firebase as possible backend technologies. Firebase Realtime Database

or MySQL can be used for data storage, while Firebase Cloud Messaging or Twilio SMS can be used for notifications.

For this implementation, Python Flask is used as the backend and HTML/CSS/JavaScript are used to develop the user interface.

3.4 Comparison

Table 3.1: Comparison of Existing and Proposed Systems

Existing Approach	**Proposed System**
Information may be distributed through different channels	Centralized web interface
Limited region-wise filtering	Region-wise schedules and filtering
Manual information access	Digital schedule display
Limited update interface	Dedicated administrative interface
Limited automation	Can be extended with alerts and notifications

Chapter 4 Methodology

4.1 System Architecture

The software implementation follows a client-server architecture.

The main components are:

1. User Interface
2. Flask Web Server
3. Schedule Data
4. Administrative Interface

The basic data flow is:

User

↓

Web Browser

↓

HTML/CSS/JavaScript

↓

Python Flask Backend

↓

Load-Shedding Schedule Data

4.2 Technologies Used

Table 4.1: Technologies Used

Technology	**Purpose**
Python	Backend programming
Flask	Web application framework
HTML	Structure of web pages
CSS	User interface styling
JavaScript	Client-side interaction
JSON	Schedule data storage in the prototype
VS Code	Development environment
Web Browser	Testing and output display

4.3 Working Procedure

The working procedure of the system is as follows:

1. The Flask application is started using Python.
2. The user accesses the application through a web browser.
3. The home page displays the available load-shedding information.
4. The user can select or view the required region and schedule.
5. The system displays the corresponding outage details.
6. The administrator can access the administrative interface.
7. New schedules can be entered or existing schedules can be updated.
8. The updated information can then be displayed to users.

4.4 Functional Modules

4.4.1 User Module

The user module provides access to load-shedding information. Users can view outage schedules and identify the expected periods of electricity unavailability.

4.4.2 Schedule Module

The schedule module stores information such as:

- Region
- Date
- Start time
- End time
- Reason for outage

4.4.3 Admin Module

The administrative module provides an interface for adding and updating schedules.

4.4.4 Region Module

The system supports multiple regions, allowing schedules to be organized according to geographical areas.

4.4.5 Notification Extension

The project specification identifies time-based alerts and SMS/push notifications as possible extensions. These features can be integrated in a future version.

Chapter 5 Implementation and Code

5.1 Project Structure

The Flask implementation is organized as follows:

```
LoadSheddingManager/
|
|-- app.py
|-- requirements.txt
|-- schedules.json
|
|-- templates/
|   |-- index.html
|   `-- admin.html
|
`-- static/
    `-- style.css
```

5.2 Flask Backend

The main backend application is implemented using Python Flask.

5.2.1 Required Libraries

```
from flask import Flask, render_template, request, jsonify
```

Listing 5.1: Required Flask Import

5.2.2 Flask Application

The Flask application is initialized using:

```
from flask import Flask, render_template, request, jsonify

```

```
app = Flask(__name__)
```

Listing 5.2: Flask Application Initialization

5.2.3 Home Page

The home page is provided through the following route:

```
@app.route("/")
def home():
    return render_template("index.html")
```

Listing 5.3: Home Page Route

5.2.4 Schedule Data

The schedule information is stored in a structured format. A sample schedule is shown below:

Listing 5.4: Sample Schedule Data

```
[
    {
        "region": "Kilnattur",
        "date": "2026-08-28",
        "start": "09:00",
        "end": "11:00",
        "reason": "Transformer maintenance"
    }
]
```

5.3 Complete Backend Code

The complete `app.py` used in the final implementation can be included below.

```
from flask import Flask, render_template, request, redirect, url_for,
    jsonify
from datetime import datetime
import json
from pathlib import Path

app = Flask(__name__)

DATA_FILE = Path("schedules.json")


# ------------------------------------------------------
# Default data
```

```
# ------------------------------------------------------------

DEFAULT_DATA = {
    "Kilnattur": [
        {
            "date": "2026-08-27",
            "start": "10:00",
            "end": "12:00",
            "reason": "Scheduled maintenance"
        },
        {
            "date": "2026-08-27",
            "start": "17:30",
            "end": "19:00",
            "reason": "Feeder maintenance"
        }
    ],

    "Chengalpattu": [
        {
            "date": "2026-08-27",
            "start": "08:30",
            "end": "10:00",
            "reason": "Line maintenance"
        },
        {
            "date": "2026-08-28",
            "start": "14:00",
            "end": "16:00",
            "reason": "Transformer work"
        }
    ],

    "Kanchipuram": [
        {
            "date": "2026-08-27",
            "start": "13:00",
            "end": "15:00",
            "reason": "Scheduled maintenance"
        }
    ]
}


# ------------------------------------------------------------
# Load schedules
# ------------------------------------------------------------
```

```
60
61  def load_schedules():
62
63      if not DATA_FILE.exists():
64
65          DATA_FILE.write_text(
66              json.dumps(DEFAULT_DATA, indent=4),
67              encoding="utf-8"
68          )
69
70      try:
71
72          with open(DATA_FILE, "r", encoding="utf-8") as file:
73              return json.load(file)
74
75      except (json.JSONDecodeError, OSError):
76
77          DATA_FILE.write_text(
78              json.dumps(DEFAULT_DATA, indent=4),
79              encoding="utf-8"
80          )
81
82          return DEFAULT_DATA
83
84
85  # ------------------------------------------------------
86  # Save schedules
87  # ------------------------------------------------------
88
89  def save_schedules(data):
90
91      with open(DATA_FILE, "w", encoding="utf-8") as file:
92
93          json.dump(
94              data,
95              file,
96              indent=4
97          )
98
99
100 # ------------------------------------------------------
101 # Home page
102 # ------------------------------------------------------
103
104 @app.route("/")
105 def home():
106
```

```
    data = load_schedules()

    regions = sorted(data.keys())

    region = request.args.get(
        "region",
        regions[0] if regions else ""
    )

    selected_date = request.args.get(
        "date",
        datetime.now().strftime("%Y-%m-%d")
    )

    schedules = data.get(region, [])

    schedules_for_date = [
        schedule
        for schedule in schedules
        if schedule["date"] == selected_date
    ]

    schedules_for_date.sort(
        key=lambda x: x["start"]
    )

    return render_template(
        "index.html",
        regions=regions,
        region=region,
        selected_date=selected_date,
        schedules=schedules_for_date
    )


# ------------------------------------------------------------
# Admin page
# ------------------------------------------------------------

@app.route("/admin", methods=["GET", "POST"])
def admin():

    data = load_schedules()

    if request.method == "POST":

        region = request.form.get(
```

```
            "region",
            ""
        ).strip()

        date = request.form.get(
            "date",
            ""
        )

        start = request.form.get(
            "start",
            ""
        )

        end = request.form.get(
            "end",
            ""
        )

        reason = request.form.get(
            "reason",
            ""
        ).strip()

        if not reason:
            reason = "Scheduled maintenance"

        # Basic validation
        if not region or not date or not start or not end:

            return render_template(
                "admin.html",
                data=data,
                error="Please fill in all required fields."
            )

        if start >= end:

            return render_template(
                "admin.html",
                data=data,
                error="End time must be later than start time."
            )

        # Create region if it does not exist
        if region not in data:

```

```
            data[region] = []

        # Add new schedule
        data[region].append(
            {
                "date": date,
                "start": start,
                "end": end,
                "reason": reason
            }
        )

        save_schedules(data)

        return redirect(
            url_for(
                "admin",
                saved="1"
            )
        )

    return render_template(
        "admin.html",
        data=data,
        saved=request.args.get("saved"),
        error=None
    )


# ------------------------------------------------------
# Delete schedule
# ------------------------------------------------------

@app.route("/delete", methods=["POST"])
def delete_schedule():

    data = load_schedules()

    region = request.form.get("region")
    index = request.form.get("index")

    if region in data:

        try:

            index = int(index)

```

```
            if 0 <= index < len(data[region]):

                data[region].pop(index)

        except (ValueError, TypeError):

            pass

    save_schedules(data)

    return redirect(
        url_for(
            "admin"
        )
    )


# ------------------------------------------------------------
# API
# ------------------------------------------------------------

@app.route("/api/schedules")
def api_schedules():

    data = load_schedules()

    region = request.args.get(
        "region"
    )

    selected_date = request.args.get(
        "date"
    )

    schedules = data.get(
        region,
        []
    )

    if selected_date:

        schedules = [
            schedule
            for schedule in schedules
            if schedule["date"] == selected_date
        ]

```

```
    return jsonify(
        {
            "region": region,
            "date": selected_date,
            "schedules": schedules
        }
    )


# ------------------------------------------------------------
# Run application
# ------------------------------------------------------------

if __name__ == "__main__":

    app.run(
        host="127.0.0.1",
        port=5000,
        debug=True
    )
```

Listing 5.5: Complete Flask Backend Code

5.4 Frontend Code

The user interface is implemented using HTML, CSS, and JavaScript.

5.4.1 HTML

The main web page is stored in:

`templates/index.html`

The complete HTML code can be included below.

```
<!DOCTYPE html>
<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">

    <title>
```

```
        Load-Shedding Time Manager
    </title>

    <link rel="stylesheet"
          href="{{ url_for('static', filename='style.css') }}">

</head>

<body>

<header>

    <div>

        <h1>
                Load-Shedding Time Manager
        </h1>

        <p>
            Rural Power Outage Management System
        </p>

    </div>

    <a class="admin-button"
       href="{{ url_for('admin') }}">

        Admin Panel

    </a>

</header>


<main>


    <!-- Search section -->

    <section class="card">

        <h2>
            Check Power Schedule
        </h2>

        <form method="GET"
              class="search-form">
```

```
59
60          <div>
61
62              <label>
63                  Region
64              </label>
65
66              <select name="region">
67
68                  {% for item in regions %}
69
70                  <option value="{{ item }}"
71                      {% if item == region %}
72                      selected
73                      {% endif %}>
74
75                      {{ item }}
76
77                  </option>
78
79                  {% endfor %}
80
81              </select>
82
83          </div>
84
85
86          <div>
87
88              <label>
89                  Date
90              </label>
91
92              <input type="date"
93                     name="date"
94                     value="{{ selected_date }}"
95                     required>
96
97          </div>
98
99
100         <button type="submit">
101
102             View Schedule
103
104         </button>
105
```

```
        </form>

    </section>



    <!-- Schedule -->

    <section class="card">

        <div class="section-header">

            <div>

                <h2>
                    {{ region }}
                </h2>

                <p>
                    Power outage schedule for
                    <strong>
                        {{ selected_date }}
                    </strong>
                </p>

            </div>


            <button onclick="enableNotification()"
                    class="notification-button">

                       Enable Alert

            </button>

        </div>


        {% if schedules %}

        <div class="schedule-container">

            {% for schedule in schedules %}

            <div class="outage-card">

                <div class="outage-time">
```

```
153
154                     {{ schedule.start }}
155
156                     {{ schedule.end }}
157
158                 </div>
159
160                 <h3>
161                     Power Unavailable
162                 </h3>
163
164                 <p>
165
166                     {{ schedule.reason }}
167
168                 </p>
169
170                 <span class="status">
171
172                     Scheduled Outage
173
174                 </span>
175
176             </div>
177
178             {% endfor %}
179
180         </div>
181
182         {% else %}
183
184         <div class="no-outage">
185
186             <div class="check">
187
188
189
190             </div>
191
192             <h3>
193                 No Scheduled Outage
194             </h3>
195
196             <p>
197                 No power outage is scheduled
198                 for this region on this date.
199             </p>
```

```
200
201         </div>
202
203         {% endif %}
204
205     </section>
206
207
208
209     <!-- Features -->
210
211     <section class="card">
212
213         <h2>
214             System Features
215         </h2>
216
217         <div class="features">
218
219             <div class="feature">
220
221
222
223                 <h3>
224                     Daily Schedule
225                 </h3>
226
227                 <p>
228                     View planned power outages.
229                 </p>
230
231             </div>
232
233
234             <div class="feature">
235
236
237
238                 <h3>
239                     Multi-Region
240                 </h3>
241
242                 <p>
243                     Select different rural regions.
244                 </p>
245
246             </div>
```

```
247
248
249             <div class="feature">
250
251
252
253                 <h3>
254                     Alerts
255                 </h3>
256
257                 <p>
258                     Browser notification support.
259                 </p>
260
261             </div>
262
263
264             <div class="feature">
265
266
267
268                 <h3>
269                     Local Storage
270                 </h3>
271
272                 <p>
273                     Schedules are stored locally.
274                 </p>
275
276             </div>
277
278         </div>
279
280     </section>
281
282
283 </main>
284
285
286 <script>
287
288 function enableNotification() {
289
290     if (!("Notification" in window)) {
291
292         alert(
293             "Notifications are not supported by this browser."
```

```
        );

        return;

    }


    Notification.requestPermission()
        .then(function(permission) {

            if (permission === "granted") {

                new Notification(
                    "Load-Shedding Manager",
                    {
                        body:
                        "Load-shedding alerts are enabled."
                    }
                );

            }

            else {

                alert(
                    "Notification permission was not granted."
                );

            }

        });

}

</script>

</body>

</html>
```

Listing 5.6: Main HTML Interface

5.4.2 CSS

The appearance of the application is controlled using:

`static/style.css`

Listing 5.7: CSS Styling Code

```
* {
    box-sizing: border-box;
}

body {
    margin: 0;
    font-family: Arial, Helvetica, sans-serif;
    background: #eef3f7;
    color: #1f2937;
}


/* Header */

header {
    background: #0f3d56;
    color: white;
    padding: 28px 7%;

    display: flex;
    justify-content: space-between;
    align-items: center;

    gap: 20px;
}

header h1 {
    margin: 0 0 8px;
    font-size: 30px;
}

header p {
    margin: 0;
    opacity: 0.9;
}


.admin-button {
    color: white;
    text-decoration: none;

    border: 1px solid rgba(255,255,255,0.5);

    padding: 11px 18px;

```

```
    border-radius: 8px;
}


/* Main */

main {
    max-width: 1100px;

    margin: 30px auto;

    padding: 0 20px;
}


/* Card */

.card {
    background: white;

    border-radius: 14px;

    padding: 25px;

    margin-bottom: 22px;

    box-shadow:
        0 5px 20px rgba(0,0,0,0.07);
}

.card h2 {
    margin-top: 0;
}


/* Forms */

.search-form {
    display: flex;

    flex-wrap: wrap;

    gap: 18px;

    align-items: end;
}

```

```
.search-form > div {
    display: flex;

    flex-direction: column;

    gap: 7px;
}

label {
    font-weight: bold;
}

input,
select {
    padding: 12px;

    border: 1px solid #cbd5e1;

    border-radius: 8px;

    font-size: 15px;
}


button {
    border: none;

    background: #0f6b8f;

    color: white;

    padding: 12px 20px;

    border-radius: 8px;

    font-weight: bold;

    cursor: pointer;
}

button:hover {
    opacity: 0.9;
}


/* Header inside card */

```

```
.section-header {
    display: flex;

    justify-content: space-between;

    align-items: center;

    gap: 15px;
}


/* Schedule */

.schedule-container {
    display: grid;

    grid-template-columns:
        repeat(auto-fit, minmax(260px, 1fr));

    gap: 18px;

    margin-top: 20px;
}


.outage-card {
    background: #fff7f7;

    border-left: 6px solid #dc2626;

    padding: 20px;

    border-radius: 10px;
}


.outage-time {
    font-size: 25px;

    font-weight: bold;

    color: #b91c1c;
}

.outage-card h3 {
    margin-bottom: 5px;
}
```

```
187
188 .outage-card p {
189     margin-bottom: 15px;
190 }
191
192
193 .status {
194     display: inline-block;
195
196     background: #fee2e2;
197
198     color: #991b1b;
199
200     padding: 6px 10px;
201
202     border-radius: 20px;
203
204     font-size: 12px;
205
206     font-weight: bold;
207 }
208
209
210 /* No outage */
211
212 .no-outage {
213     text-align: center;
214
215     background: #f0fdf4;
216
217     padding: 35px;
218
219     border-radius: 10px;
220 }
221
222 .check {
223     font-size: 45px;
224
225     color: #16a34a;
226 }
227
228
229 /* Features */
230
231 .features {
232     display: grid;
233
```

```
    grid-template-columns:
        repeat(auto-fit, minmax(200px, 1fr));

    gap: 15px;
}

.feature {
    background: #f8fafc;

    padding: 18px;

    border-radius: 10px;
}

.feature h3 {
    margin-bottom: 5px;
}


/* Admin */

.admin-form {
    display: grid;

    grid-template-columns:
        repeat(2, 1fr);

    gap: 18px;
}

.admin-form > div {
    display: flex;

    flex-direction: column;

    gap: 7px;
}

.full-width {
    grid-column: 1 / -1;
}

.save-button {
    grid-column: 1 / -1;
}


```

```
/* Messages */

.success-message {
    background: #dcfce7;

    color: #166534;

    padding: 15px;

    border-radius: 10px;

    margin-bottom: 20px;

    font-weight: bold;
}

.error-message {
    background: #fee2e2;

    color: #991b1b;

    padding: 15px;

    border-radius: 10px;

    margin-bottom: 20px;
}


/* Table */

.region-title {
    margin-top: 28px;
}

.table-container {
    overflow-x: auto;
}

table {
    width: 100%;

    border-collapse: collapse;
}

th,
td {
```

```
    padding: 12px;

    border-bottom: 1px solid #e2e8f0;

    text-align: left;
}

th {
    background: #f8fafc;
}


.delete-button {
    background: #dc2626;

    padding: 8px 12px;

    font-size: 12px;
}


/* Responsive */

@media (max-width: 700px) {

    header {
        flex-direction: column;

        align-items: flex-start;
    }

    .admin-form {
        grid-template-columns: 1fr;
    }

    .full-width,
    .save-button {
        grid-column: auto;
    }

    .section-header {
        flex-direction: column;

        align-items: flex-start;
    }

}
```

5.5 Administrative Interface

The administrative page is implemented using:

`templates/admin.html`

The administrator can enter schedule information including the region, date, start time, end time, and reason for the outage.

```
<!DOCTYPE html>
<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">

    <title>
        Load-Shedding Admin Panel
    </title>

    <link rel="stylesheet"
          href="{{ url_for('static', filename='style.css') }}">

</head>

<body>

<header>

    <div>

        <h1>
                    Admin Panel
        </h1>

        <p>
            Electricity Board Schedule Management
        </p>

    </div>

    <a class="admin-button"
       href="{{ url_for('home') }}">

```

```
        User Dashboard

    </a>

</header>


<main>


    {% if saved %}

    <div class="success-message">

            Schedule saved successfully.

    </div>

    {% endif %}


    {% if error %}

    <div class="error-message">

            {{ error }}

    </div>

    {% endif %}



    <!-- Add schedule -->

    <section class="card">

        <h2>
            Add New Outage Schedule
        </h2>

        <form method="POST"
              class="admin-form">


            <div>

```

```
                <label>
                    Region
                </label>

                <input type="text"
                       name="region"
                       placeholder="Example: Kilnattur"
                       required>

            </div>


            <div>

                <label>
                    Date
                </label>

                <input type="date"
                       name="date"
                       required>

            </div>


            <div>

                <label>
                    Start Time
                </label>

                <input type="time"
                       name="start"
                       required>

            </div>


            <div>

                <label>
                    End Time
                </label>

                <input type="time"
                       name="end"
                       required>
```

```
133
134             </div>
135
136
137             <div class="full-width">
138
139                 <label>
140                     Reason
141                 </label>
142
143                 <input type="text"
144                        name="reason"
145                        placeholder="Scheduled maintenance">
146
147             </div>
148
149
150             <button type="submit"
151                     class="save-button">
152
153                 Save Schedule
154
155             </button>
156
157         </form>
158
159     </section>
160
161
162
163     <!-- Existing schedules -->
164
165     <section class="card">
166
167         <h2>
168             Existing Schedules
169         </h2>
170
171
172         {% for region, schedules in data.items() %}
173
174         <h3 class="region-title">
175
176                     {{ region }}
177
178         </h3>
179
```

```
180
181         {% if schedules %}
182
183         <div class="table-container">
184
185             <table>
186
187                 <thead>
188
189                     <tr>
190
191                         <th>
192                             Date
193                         </th>
194
195                         <th>
196                             Time
197                         </th>
198
199                         <th>
200                             Reason
201                         </th>
202
203                         <th>
204                             Action
205                         </th>
206
207                     </tr>
208
209                 </thead>
210
211
212                 <tbody>
213
214                 {% for schedule in schedules %}
215
216                 <tr>
217
218                     <td>
219                         {{ schedule.date }}
220                     </td>
221
222                     <td>
223                         {{ schedule.start }}
224
225                         {{ schedule.end }}
226                     </td>
```

```
227
228                     <td>
229                         {{ schedule.reason }}
230                     </td>
231
232                     <td>
233
234                         <form method="POST"
235                               action="{{ url_for('delete_schedule') }}">
236
237                             <input type="hidden"
238                                    name="region"
239                                    value="{{ region }}">
240
241                             <input type="hidden"
242                                    name="index"
243                                    value="{{ loop.index0 }}">
244
245                             <button type="submit"
246                                     class="delete-button">
247
248                                 Delete
249
250                             </button>
251
252                         </form>
253
254                     </td>
255
256                 </tr>
257
258                 {% endfor %}
259
260                 </tbody>
261
262             </table>
263
264         </div>
265
266         {% else %}
267
268         <p>
269             No schedules available.
270         </p>
271
272         {% endif %}
273
```

```
274         {% endfor %}
275
276     </section>
277
278 </main>
279
280 </body>
281
282 </html>
```

Listing 5.8: Administrative Interface Code

Chapter 6 Output Snapshots

This chapter presents the output obtained after running the Flask application in Visual Studio Code and accessing it through a web browser.

6.1 Main Dashboard

The main dashboard displays the load-shedding information available to the user.

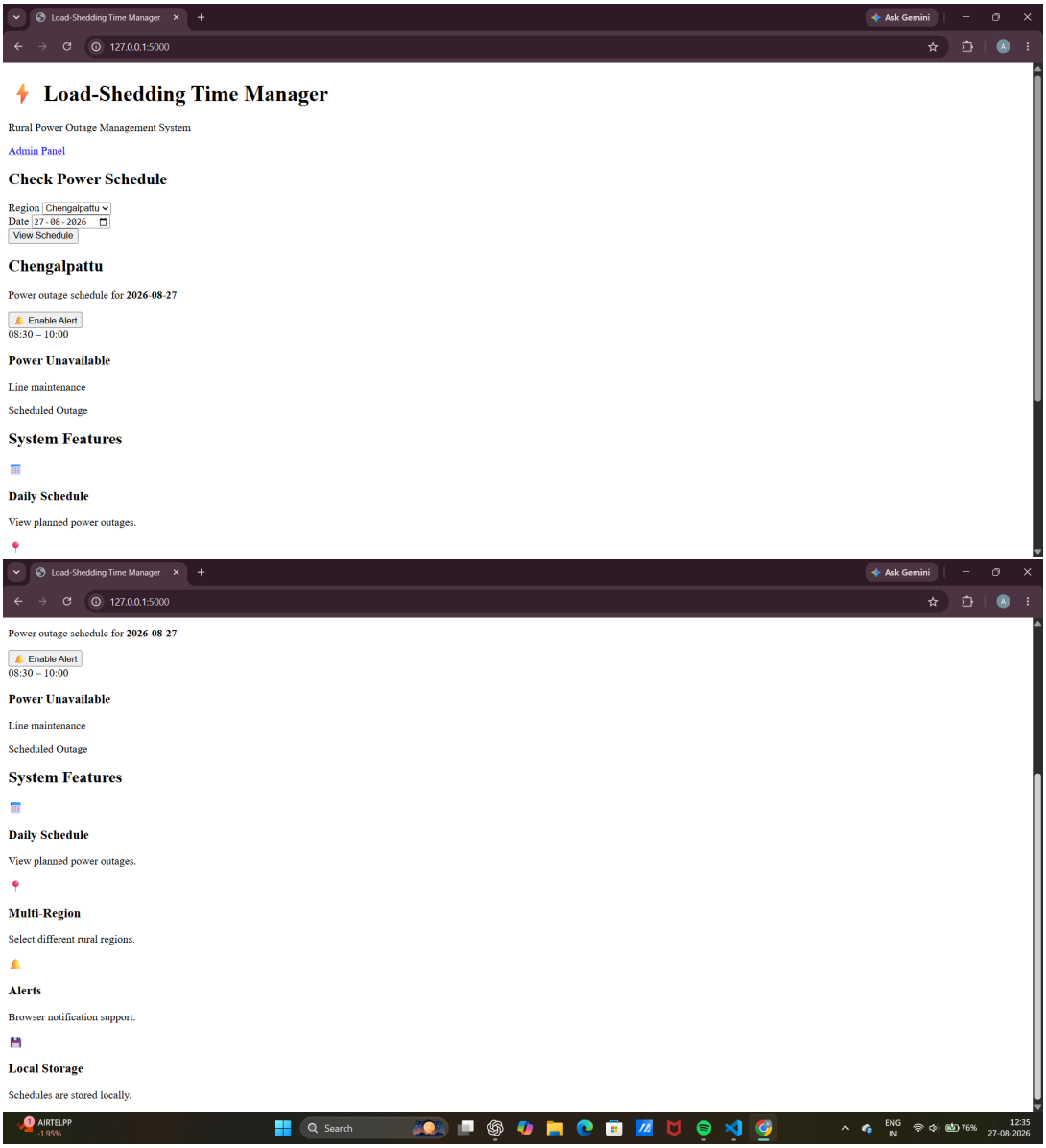


Figure 6.1: Load-Shedding Time Manager Main Dashboard

6.2 Administrative Panel

The administrative panel is used to enter and update outage information.

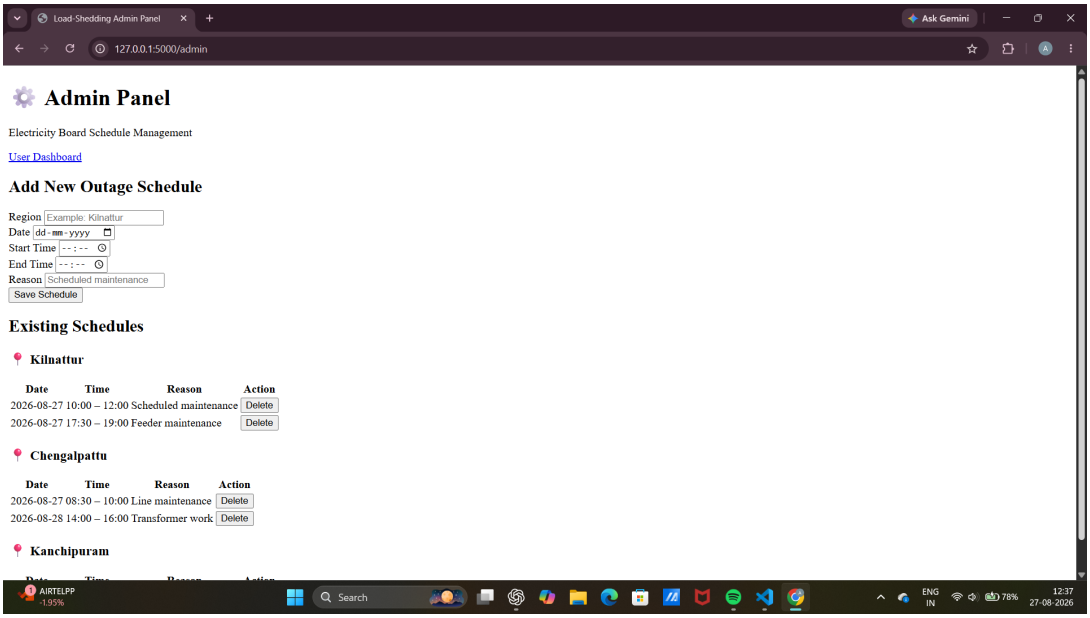


Figure 6.2: Administrative Panel

6.3 Schedule Added Successfully

The following snapshot demonstrates that a new schedule has been entered successfully.

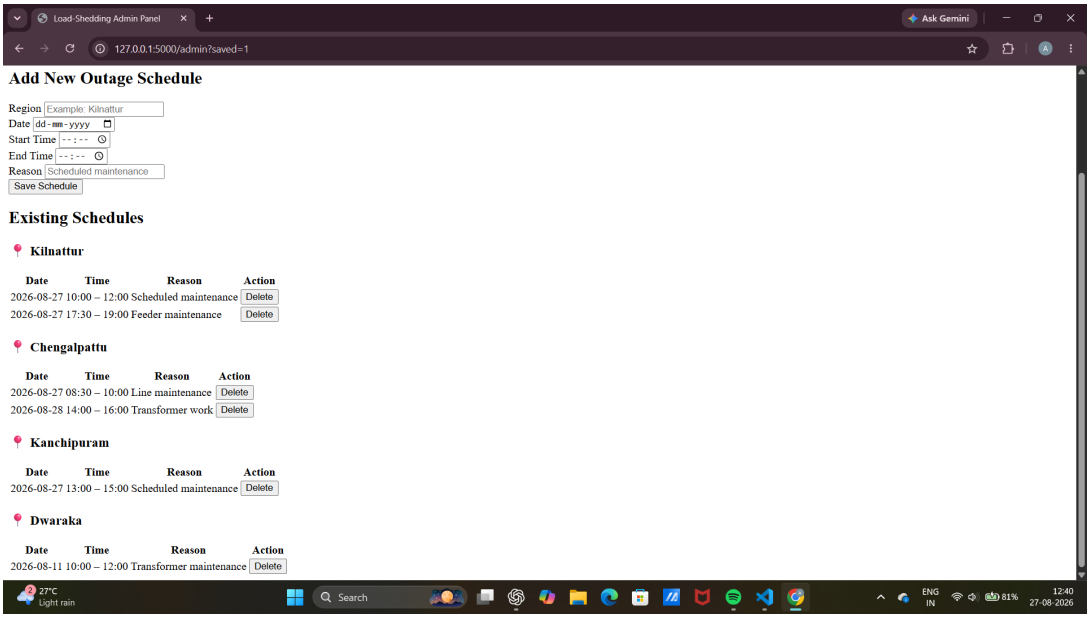


Figure 6.3: Updated Load-Shedding Schedule

6.4 Running Flask Server

The Flask server can also be shown running in the VS Code terminal.

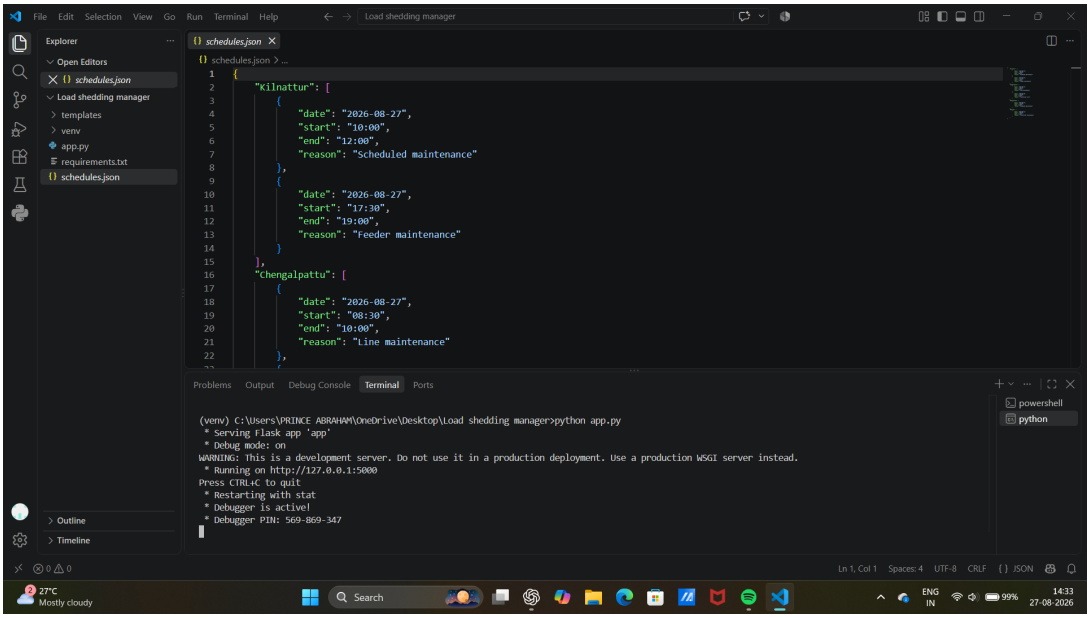


Figure 6.4: Flask Server Running in VS Code Terminal

Chapter 7 Result and Analysis

7.1 Result

The implemented Load-Shedding Time Manager successfully demonstrates the basic functionality required for managing and displaying scheduled power outages.

The Flask-based application provides a browser-based interface for accessing load-shedding information. The system can display schedule information and provide an administrative interface for managing outage data.

The prototype also demonstrates the possibility of organizing schedules by region.

7.2 Functional Testing

The following tests were performed on the application.

Table 7.1: Functional Test Results

Test	**Expected Result**	**Status**
Open home page	Dashboard displayed	Pass
View schedule	Correct schedule displayed	Pass
Select region	Region-specific information displayed	Pass
Open admin panel	Admin interface displayed	Pass
Add schedule	Schedule stored/displayed	Pass
Run Flask server	Application accessible in browser	Pass

7.3 Analysis

The project demonstrates that a lightweight Flask web application can be used to provide centralized access to load-shedding information.

The main advantage of the prototype is its simple architecture. Users require only a web browser to access the interface, while administrators can manage schedule information through

the administrative interface.

Region-wise scheduling can improve the relevance of the information provided to users. The project can also be extended to include automated notifications, database storage, offline caching, and hardware-based displays.

The current implementation is a software prototype. Advanced notification services and optional hardware components are not required for the basic Flask demonstration.

Chapter 8 Conclusion

The **Load-Shedding Time Manager for Rural Areas** was developed as a web-based solution for managing and displaying scheduled electricity outages.

The Flask-based implementation successfully demonstrates the basic operation of the system. Users can access the web interface and view load-shedding schedules, while an administrative interface provides a method for entering and updating schedule information.

The project addresses the need for organized and accessible load-shedding information and can help users plan activities such as agriculture, education, household work, and small-business operations around expected power interruptions.

The system provides a foundation for future improvements including database integration, automated SMS or push notifications, offline caching, and optional RTC/LCD-based hardware displays.

Overall, the project demonstrates the practical use of Python Flask and web technologies to develop a simple and extensible load-shedding information management system.

Future Scope

The following improvements can be considered for future versions:

1. Integration with a real-time database such as MySQL or Firebase.
2. Automatic SMS notifications before scheduled outages.
3. Push notifications through a mobile application.
4. Offline caching of previously available schedules.
5. Development of a dedicated Android or mobile application.
6. Integration of an RTC module and LCD display for locations with limited internet connectivity.
7. Addition of user authentication and administrator authorization.
8. Automatic synchronization with electricity-board schedule data.

References

1. Project specification: *Load-Shedding Time Manager for Rural Areas*, Internship Projects document.

2. Python Software Foundation, *Python Documentation*.

3. Flask Documentation, *Flask Web Development Framework*.

4. HTML, CSS and JavaScript documentation and learning resources used during implementation.